

20

Lecture 20

Memory Management II: Paging Fundamentals

by Brijesh Shukla

C07: Operating Systems

Join the lecture online on your dashboard

Today's Agenda

- Review of Contiguous Allocation Issues
- Introduction to Paging
- Page Table Structure and Entries
- Address Translation Mechanics
- Advantages and Disadvantages
- Advanced Topics: Multi-Level Tables



Problems with Contiguous Allocation

✖ External Fragmentation

- Free memory split into **small holes**
- Enough total space, but **not contiguous**
- Example: $50\text{KB} + 30\text{KB} + 40\text{KB} \neq 100\text{KB}$

📐 Limited Flexibility

- Process size must be **known in advance**
- ❌ Hard to **grow memory dynamically**

📈 Poor Memory Utilization

- Free space wasted due to fragmentation
- Leads to inefficient RAM usage

🔄 Compaction Overhead

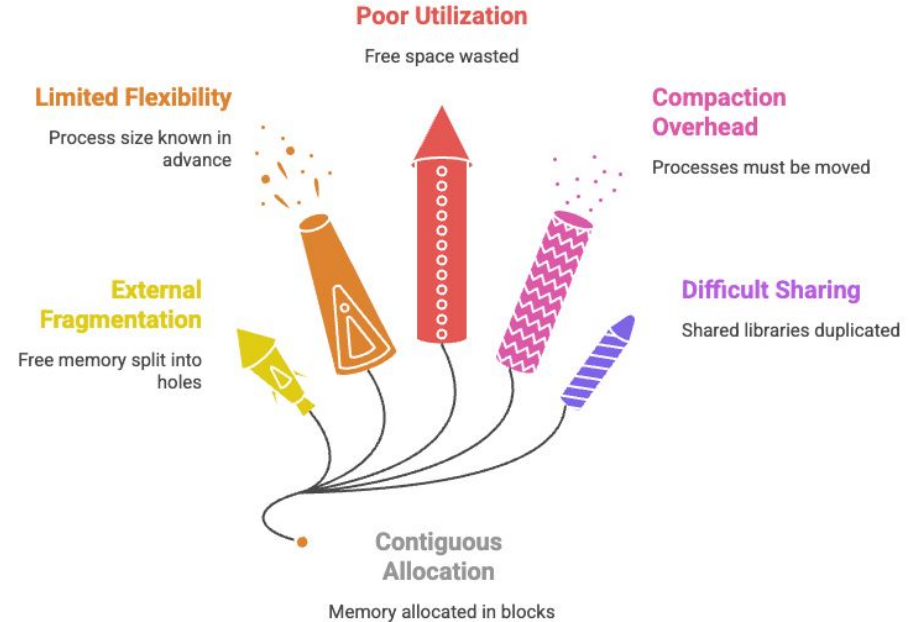
- Processes must be **moved**
- System pause required
- ⌚ Costly: **$O(n)$** time complexity

📖 Difficult Memory Sharing

- Shared libraries get **duplicated**
- ❌ Wastes memory

🚀 Key Takeaway

Contiguous allocation is inefficient and inflexible. Need a non-contiguous memory allocation technique (Paging)



Introduction to Paging

What is Paging?

- **Paging** is a memory management technique
- Allows a process to be stored in **non-contiguous physical memory**
- Removes the need for contiguous allocation

🧠 Core Idea

- **Physical Memory** → **Frames** (fixed-size blocks)
- **Logical Memory** → **Pages** (same size as frames)
-  Any page can be placed in **any free frame**

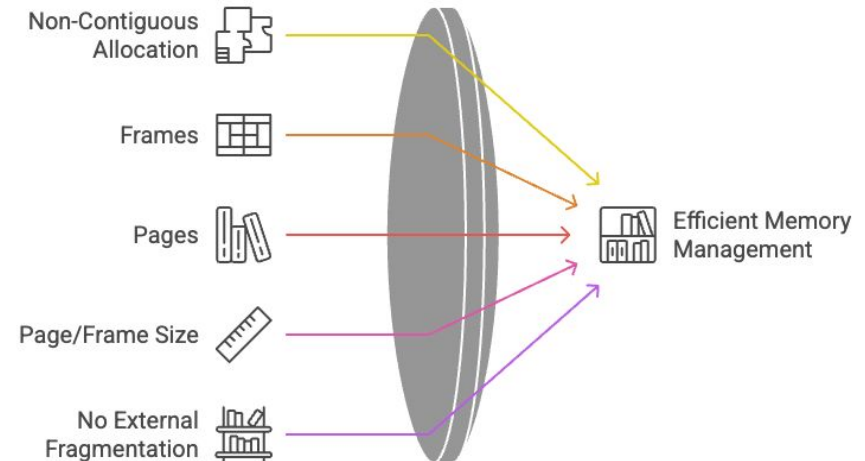
📦 Page / Frame Size

- Fixed size, **power of 2**
- Common sizes: 4 KB (most common). 8 KB. 4 MB
- Power of 2 → **easy address calculation**

★ Key Advantage

-  **No External Fragmentation**
-  Better memory utilization
-  Flexible process allocation

The Power of Paging



Paging – Visual Representation

🧠 Logical Memory (Process)

- Divided into **fixed-size pages**
- Pages are **contiguous**
- Ordered: Page 0, Page 1, Page 2, ...

💾 Physical Memory (RAM)

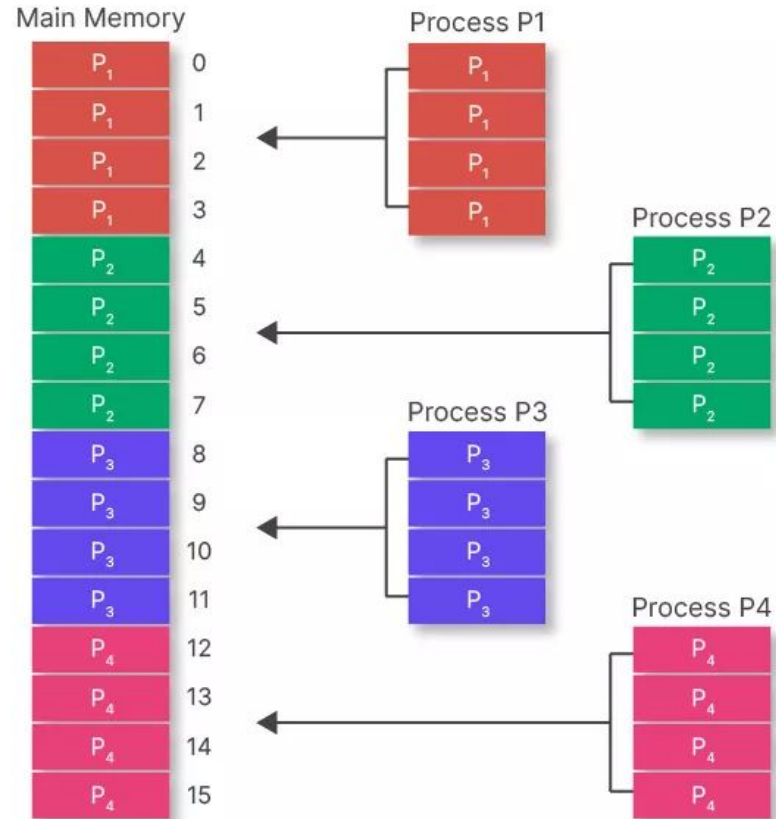
- Divided into **fixed-size frames**
- Frames may be **scattered**
- Some frames may be **free or used by OS**

🔗 Page–Frame Mapping

- Pages can be placed in **any available frame**
- Mapping maintained using a **Page Table**
- Enables **non-contiguous allocation**

★ Key Takeaways

-  Pages → contiguous (logical view)
-  Frames → non-contiguous (physical view)
-  No external fragmentation
-  Efficient memory utilization



Frame Table & Page Table

1. Frame Table (Maintained by OS)

One entry **per physical frame**

Tracks **allocated / free** frames

Stores:

Frame number

Status (Free / Used)

Process ID (owner)

 **Purpose:** Efficient physical memory management

Page 0
Page 1
Page 2
Page 3

Virtual
Memory

0	Frame 1
1	Frame 4
2	Frame 3
3	Frame 7

Page Table

0	
1	Page 0
2	
3	Page 2
4	Page 1
5	
6	
7	Page 3

Physical Memory

2. Page Table (One per Process)

One entry **per logical page**

Maps **Page** → **Frame**

Stored in **main memory**

Base address stored in **PCB**

 **Purpose:** Address translation

Page Table Entry (PTE) Structure

PTE – What It Contains

Frame Number	Valid Bit	Protect (R/W/X)	Dirty Bit	Referenced Bit	Cache Disable
--------------	-----------	-----------------	-----------	----------------	---------------

Field Overview

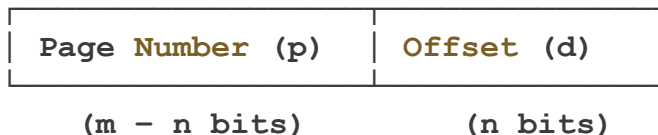
- **Frame Number**
 - Size depends on RAM (e.g., 18 bits for 1GB RAM, 4KB frames)
- **Valid / Invalid Bit**
 - 1 → Page present in memory
 - 0 → Page fault occurs
- **Protection Bits (R / W / X)**
 - Example: 100 = Read-only, 110 = Read-Write
- **Dirty (Modified) Bit**
 - Used during page replacement
- **Referenced Bit**
 - Helps replacement algorithms (LRU, Clock)
- **Cache Disable Bit**
 - Used for memory-mapped I/O

Field	Description
 Frame Number	Physical frame holding the page
 Valid / Invalid Bit	Page present in memory
 Protection Bits (R / W / X)	Access permissions
 Dirty (Modified) Bit	Page modified
 Referenced Bit	Set when page is accessed
 Cache Disable Bit	Disables caching

Address Translation – The Big Picture

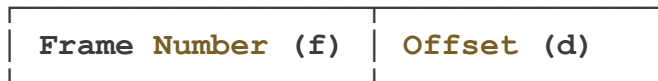
Logical Address

- Process generates a **logical address (m)**
- Split into **Page Number + Offset**



Physical Address

- Formed after page table lookup
- **Frame Number replaces Page Number**
- **Offset stays the same**



Key Rule:

- 👉 Only **page** → **frame** is translated
- 👉 **Offset** is unchanged

Example

- Logical address space = $2^{16} = 64$ KB
- Page size = $2^{10} = 1$ KB
- Number of pages = **64**
- Page number bits = **6**
- Offset bits = **10**

Logical **Address** (**16 bits**)

→ Page Number: **6** bits

→ Offset: **10** bits

Address Translation – Step by Step

Big Flow

CPU

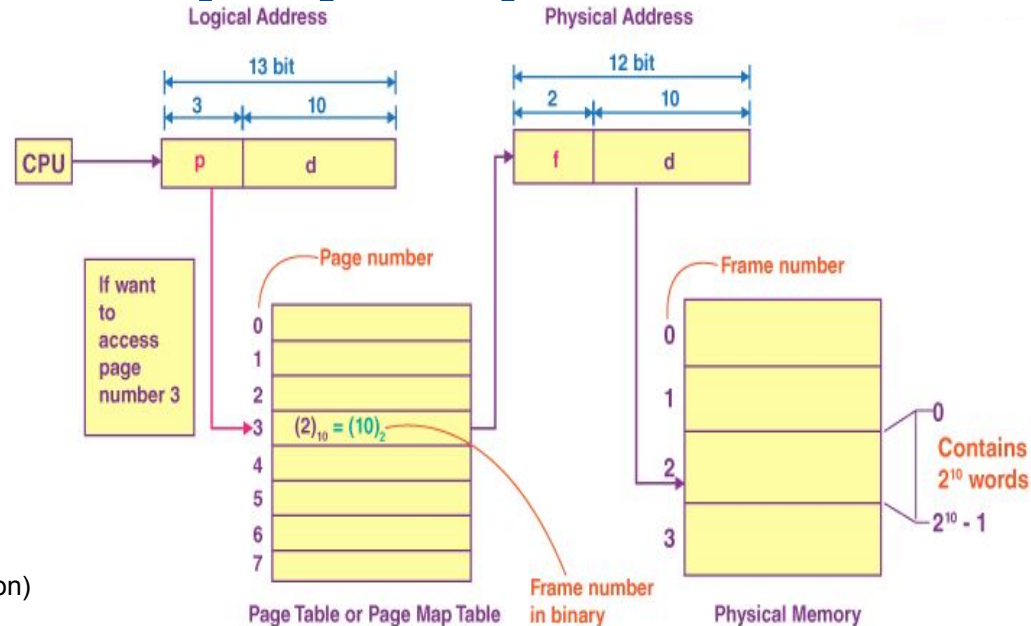
Logical Address (p, d)

↓
Page Table Lookup (p → f)

↓
Physical Address (f, d)

Step-by-Step Translation

1. **Extract Page Number (p)**
 - $p = \text{Logical Address bits} - d$
2. **Extract Offset (d)**
 - $d = \log_2(\text{Page Size})$
3. **Page Table Lookup**
 - Use p to index page table
 - Check **Valid bit**
 -  Invalid → **Page Fault** (OS intervention)
 -  Valid → Get frame number f
4. **Form Physical Address**
 - $\text{Physical Address} = (f \times \text{Page Size}) + d$
5. **Access Memory**
 - CPU reads/writes at computed physical address



Address Translation Example

 **Given:** Logical Address Space: 32 bytes (2^5). Physical Address Space: 128 bytes (2^7). Page / Frame Size: 8 bytes (2^3)

Derived Values

- **Pages:** $32 / 8 = 4$ pages. **Frames:** $128 / 8 = 16$ frames. **Page bits:** 2 bits. **Offset bits:** 3 bits. **Frame bits:** 4 bits

Page Table

Page 0 → Frame 5

Page 1 → Frame 9

Page 2 → Frame 3

Page 3 → Frame 7

Translate Logical Address = 13

1. **Binary:**

- 13 = 01101

2. **Split (Page | Offset):**

01 | 101

p d

- Page = 1. Offset = 5

3. **Page Table Lookup:**

- Page 1 → Frame 9 (1001)

4. **Physical Address:**

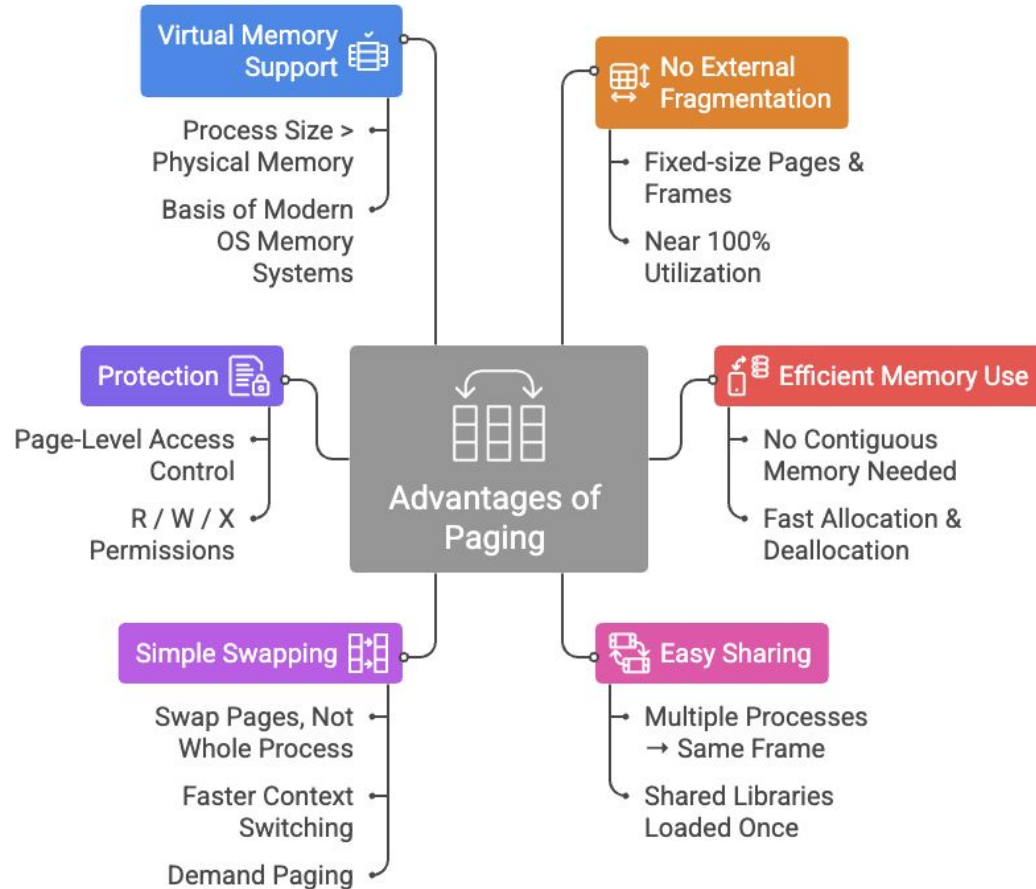
Frame | Offset

1001 | 101

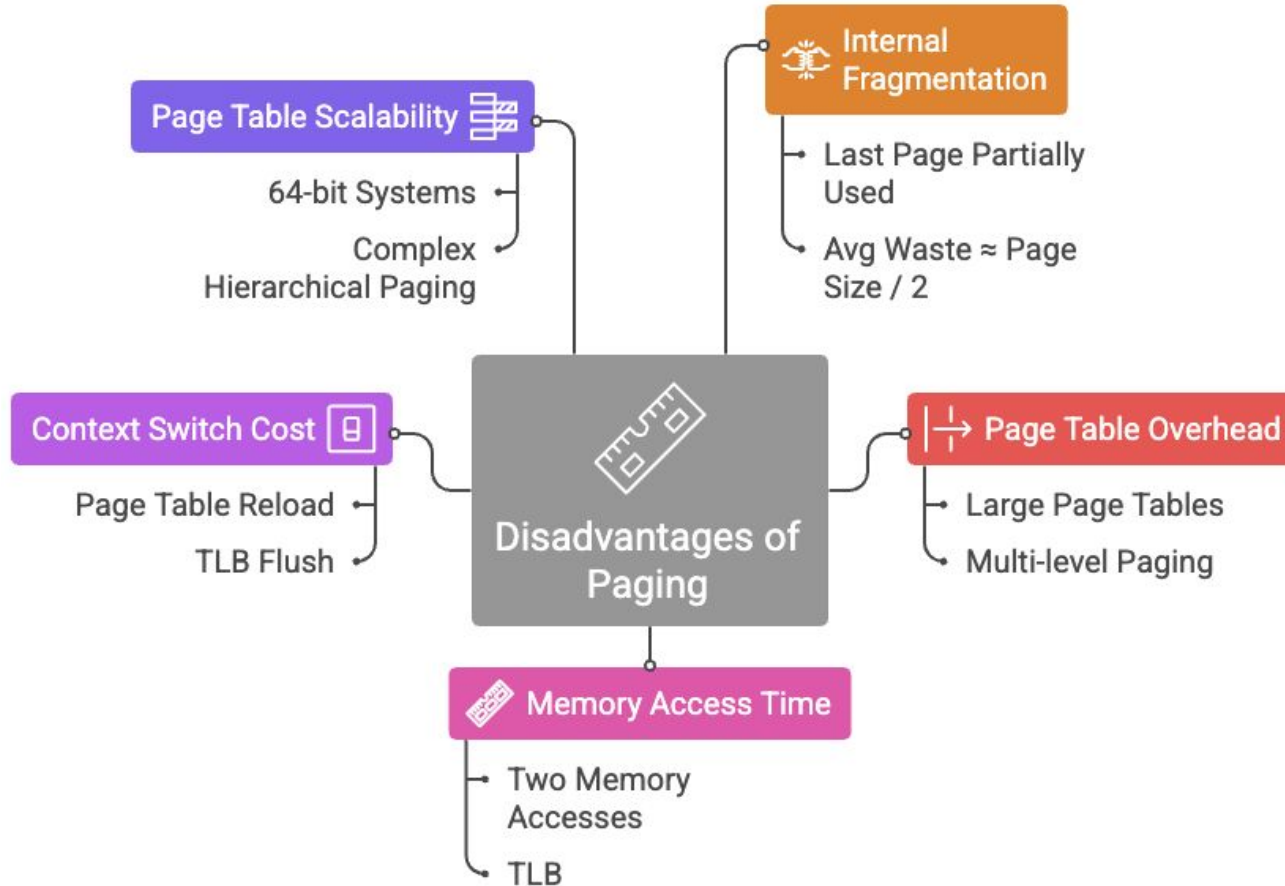
- Binary: 1001101
- Decimal: 77

 **Verification:** Frame 9 start = $9 \times 8 = 72$. $72 + 5 = 77$ ✓

Advantages of Paging



Disadvantages of Paging



Page Table Storage Problem

⚠ The Challenge (32-bit System)

- Logical address space: **4 GB (2^{32})**
- Page size: **4 KB (2^{12})**
- Pages for process: $2^{20} \approx 1$ million
- PTE size: **4 bytes**
- **Page table size = 4 MB per process**
- 📌 **100 processes → 400 MB only for page tables!**

💣 Even Worse in 64-bit Systems

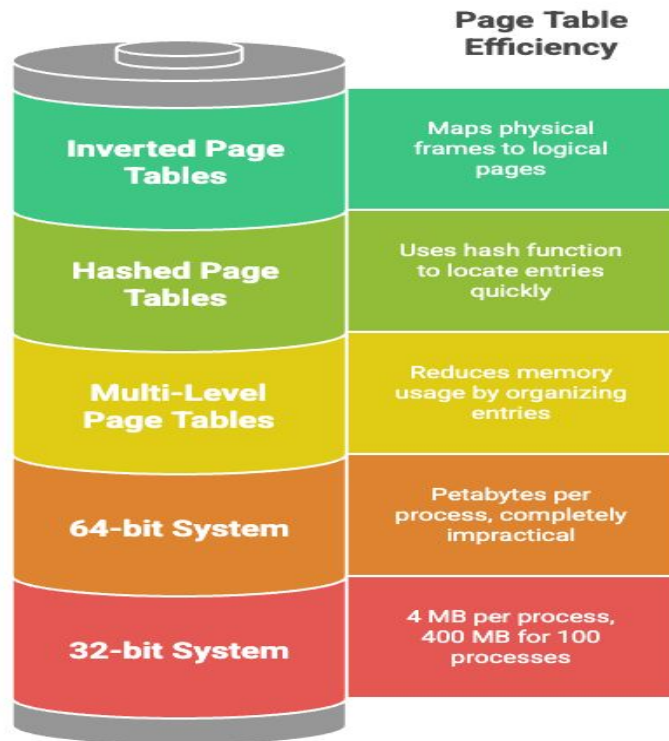
- Pages $\approx 2^{52}$
- Page table size → **Petabytes per process**
- **❌ Completely impractical**

🧠 Why This is a Problem

- Wastes huge memory
- Most page table entries unused
- Poor scalability

✅ Practical Solutions

- **Multi-Level (Hierarchical) Page Tables**
- **Hashed Page Tables**
- **Inverted Page Tables**



Multi-Level Paging – Concept

💡 **Core Idea:** Split a huge page table into smaller tables. Allocate page tables only when needed

📁 Two-Level Paging Structure

- **Outer Page Table** (Page Directory)
- **Inner Page Tables** (Second level)

🧠 Logical Address Format

P1 (10b)	P2 (10b)	Offset (12b)
----------	----------	--------------

Page Dir
Index

Page Table
Index

Byte in
Page

✅ Why It Works

- Inner page tables created **on demand**
- Unused address space → **no page table**
- Typical processes use **< 1%** of address space

★ Key Advantage

- 🚀 **Massive memory savings**
- Scales well for **32-bit & 64-bit systems**



Two-Level Paging Example

⚙️ **Configuration:** Logical address: 32 bits. **Page size:** 4 KB (2^{12}). **PTE size:** 4 bytes

🧠 Address Breakdown

P1 (10b)	P2 (10b)	d (12b)
Page Dir	Page Table	Offset

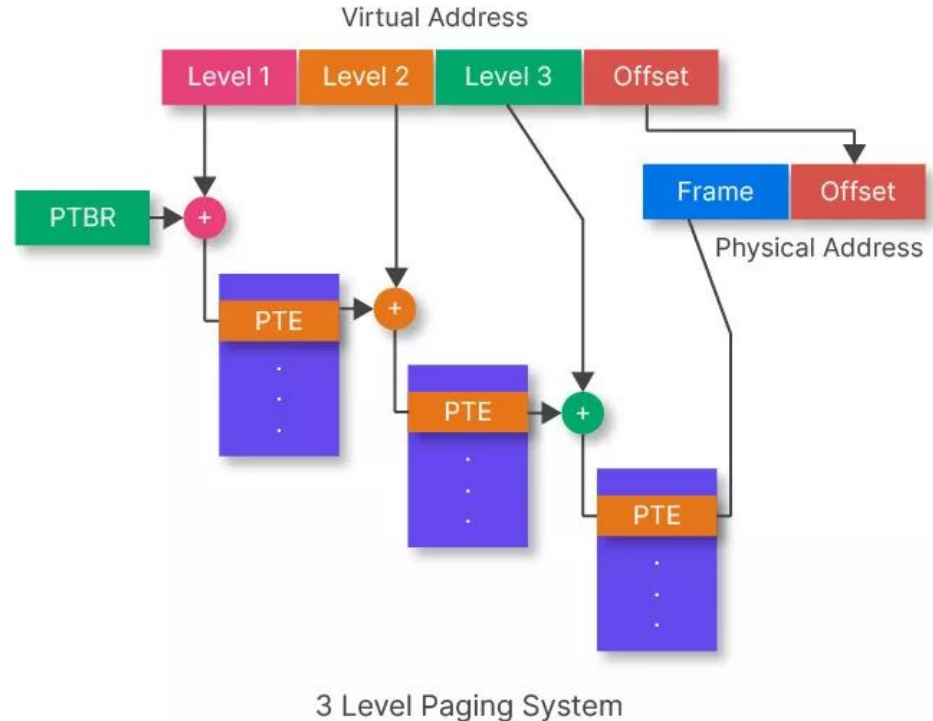
🔄 Translation Steps

1. **Index P1** → Outer Page Table
 - Get base address of inner page table
2. **Index P2** → Inner Page Table
 - Get **frame number**
3. **Frame + Offset**
 - Form **physical address**

💾 Memory Savings

- **Single-level paging:**
 - Always **4 MB** per process
- **Two-level paging (≈1% usage):**
 - Outer table: **4 KB**
 - Inner tables: **10 × 4 KB = 40 KB**
 - **Total = 44 KB**

✅ ~99% memory saved



Shared Pages

💡 Core Idea

- Multiple processes share the same physical frame
- Commonly used for **read-only code & libraries**
- Saves memory, improves cache usage

🧠 How It Works

Process P1 PT Shared Frame Process P2 PT
Pg1 → Fr15 → Fr15 ← Pg1 → Fr15

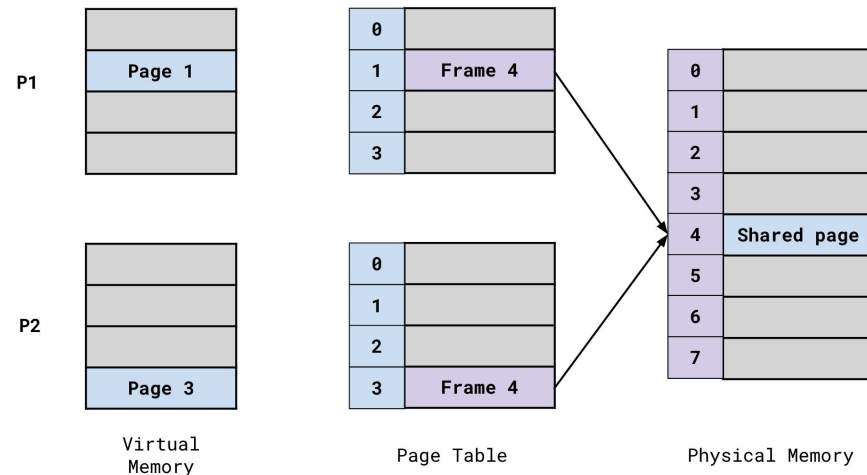
- Different page tables → **same frame**
- OS maps identical code pages to one frame

🧩 Use Cases

- **Shared Libraries:** libc, OpenGL (load once, map many)
- **Shared Code:** Multiple instances of same program
- **Shared Data:** IPC, shared-memory databases

✅ Requirements

- Pages must be **reentrant** (read-only)
- Same logical address **or**
- **Position-independent code (PIC)**



Page Size Selection

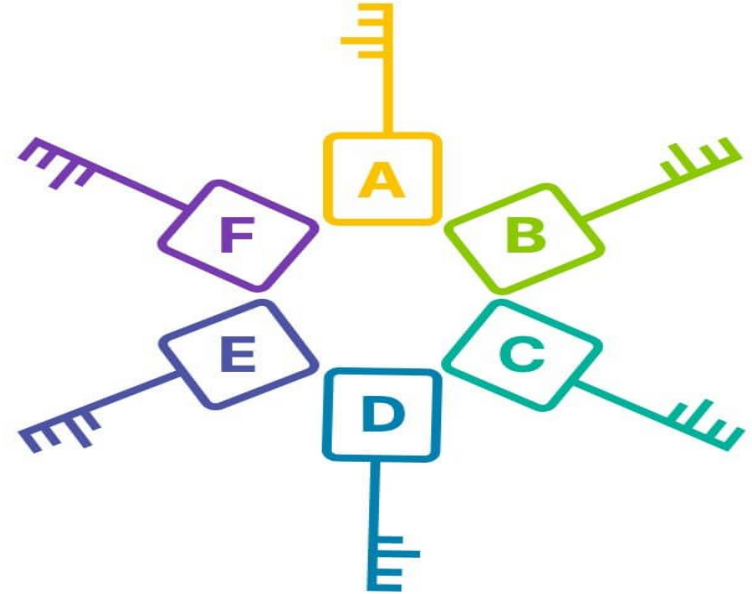
Characteristic	Small Pages (1KB, 2KB)	Large Pages (2MB, 1GB)
 Internal Fragmentation	Less	More
 Memory Utilization	Better	Worse
 Protection	Finer-grained	Coarser
 Page Tables	Large	Smaller
 TLB Misses	More	Fewer
 Management Overhead	Higher	Lower
 Access Efficiency	Less efficient	More efficient
 Memory Waste	Less	More
 Swap/Load Speed	Faster	Slower

Why Page Size Matters

- Affects **memory usage**, **performance**, and **overhead**
- OS must balance **fragmentation** vs **speed**

Key Takeaways

-  Paging Fundamentals
-  Address Translation
-  Page Table Entry (PTE)
-  Performance Optimization
-  Trade-offs



Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.